Name:**Elvis Wachira** Course:**Bsc. Computer Science**
Reg/no**: SCT211-0010/2025** email: **elvis.michinji@students.jkuat.ac.ke**

ASSIGNMENT BIT 2109: **OBJECT ORIENTED PROGRAMMING 1**

1. Explain concept of operator overloading

Operator overloading is a compile-time polymorphism feature in C++ that allows programmers to redefine the way operators work for user-defined data types (classes and structures). In essence, it enables standard operators such as +, -, *, ==, and others to be given additional meanings when applied to objects of a class, making code readable.

Characteristics of Operator Overloading:

1. The operator keyword followed by the operator symbol is used to define the overloaded function.
2. The basic meaning of the operator is not changed — only its behavior for the new types is defined.
3. Operator precedence and associativity remain unchanged.
4. Certain operators cannot be overloaded: scope resolution (::), member selector (.), sizeof, and the ternary operator (?:).
5. At least one operand must be a user-defined type.

General Syntax:

```
return_type operator op (argument_list)
{
    // function body
}
```

Operator overloading improves code readability and allows user-defined types to behave like built-in types, which is a fundamental aspect of writing clean and expressive C++ programs

2.Discuss the concept of:

a) Unary Operator Overloading

Unary operator overloading refers to overloading operators that operate on a single operand. Common unary operators include the increment (++), decrement (--), negation (-), logical NOT (!), and the dereference (*) operator.

Example program

The following program demonstrates overloading the unary minus (-) operator to negate the values of a Distance object:

```
#include <iostream>
using namespace std;

class Distance {
    private:
        int feet;
        float inches;

    public:
        Distance(int f = 0, float i = 0.0) : feet(f), inches(i) {}

        // Overloading the unary minus operator
        Distance operator- () {
            feet   = -feet;
            inches = -inches;
            return Distance(feet, inches);
        }

        void display() {
            cout << "Feet: " << feet << ", Inches: " << inches << endl;
        }
};

int main() {
    Distance D1(11, 10.0);
    Distance D2;

    cout << "Original Distance: ";
    D1.display();

    D2 = -D1;   // Unary minus applied to D1

    cout << "Negated Distance:  ";
    D2.display();

    return 0;
}
```

b) Binary operator overloading

Binary operator overloading refers to overloading operators that work on two operands. Common binary operators include addition (+), subtraction (-), multiplication (*), equality (==), less-than (<), and many others.

Program Example — Binary Plus Operator

The following program demonstrates overloading the binary + operator to add two Complex number objects:

```
#include <iostream>
using namespace std;

class Complex {
    private:
        float real;
        float imag;

    public:
        Complex(float r = 0, float i = 0) : real(r), imag(i) {}

        // Overloading the binary + operator
        Complex operator+ (const Complex& c) {
            return Complex(real + c.real, imag + c.imag);
        }

        // Overloading the binary == operator
        bool operator== (const Complex& c) {
            return (real == c.real && imag == c.imag);
        }

        void display() {
            if (imag >= 0)
                cout << real << " + " << imag << "i" << endl;
            else
                cout << real << " - " << -imag << "i" << endl;
        }
};

int main() {
    Complex C1(3.5, 2.0);
    Complex C2(1.5, -4.0);
    Complex C3;

    cout << "C1 = "; C1.display();
    cout << "C2 = "; C2.display();

    C3 = C1 + C2;   // Binary + applied
    cout << "C1 + C2 = "; C3.display();

    if (C1 == C2)
        cout << "C1 and C2 are equal." << endl;
    else
        cout << "C1 and C2 are NOT equal." << endl;

    return 0;
}
```

3. Give suitable program example to demonstate constructor overloading

Constructor overloading is the ability to define multiple constructors within the same class, each having a different parameter list (different number, type, or order of parameters).

Program Example — Constructor Overloading

The following program demonstrates constructor overloading using a Rectangle class that can be initialized in three different ways:

```
#include <iostream>
using namespace std;
```

```
class Rectangle {
    private:
        float length;
        float width;

    public:
        // Constructor 1: Default constructor (no arguments)
        Rectangle() {
            length = 0;
            width  = 0;
            cout << "Default constructor called." << endl;
        }

        // Constructor 2: Parameterized constructor (two arguments)
        Rectangle(float l, float w) {
            length = l;
            width  = w;
            cout << "Parameterized constructor called." << endl;
        }

        // Constructor 3: Single-argument constructor (square)
        Rectangle(float side) {
            length = side;
            width  = side;
            cout << "Single-argument constructor called." << endl;
        }

        // Constructor 4: Copy constructor
        Rectangle(const Rectangle& r) {
            length = r.length;
            width  = r.width;
            cout << "Copy constructor called." << endl;
        }

        float area() {
            return length * width;
        }

        void display() {
            cout << "Length = " << length
                 << ", Width = " << width
                 << ", Area = " << area() << endl;
        }
};

int main() {
    // Using Constructor 1 – default
    Rectangle R1;
    R1.display();

    // Using Constructor 2 – length and width
    Rectangle R2(10.0, 5.0);
    R2.display();

    // Using Constructor 3 – square
    Rectangle R3(7.0);
    R3.display();

    // Using Constructor 4 – copy of R2
    Rectangle R4(R2);
    R4.display();

    return 0;
}
```